

From Stateless Execution to Human-Governed AI Repair

The Evolution of AI Debugger Pro, Versions 0-6

T. R. Bentley

System Evolution Case Study

September 2026

Research focus

How a small deterministic code-checking utility evolved into a security-conscious, multi-language, AI-assisted repair workspace while preserving explicit human authority over source-code modification.

Scope note. This paper documents architectural evolution rather than presenting every phase as a formal semantic-version release. Versions 0-4 correspond to the project's original documented evolution; later engineering phases extend that lineage through the current human-governed repair, restricted-execution, interface, demonstration, and release work.

Repository: Tybent18/ai-debugger-pro

Primary branch: master

License: MIT

Abstract

AI Debugger Pro began as a lightweight Python debugging assistant: parse source code, reject syntax failures, execute valid input, and return captured output. That initial model was deliberately simple and stateless. Across successive versions, the system acquired LLM-assisted interpretation, multi-language execution, a desktop interface, persistent execution history, code-diff tracking, approval-gated repair, automatic post-repair verification, restricted Docker execution, exception-state capture, bounded project context, CLI and editor integration, a redesigned IDE-style workspace, deterministic product demonstrations, and packaged desktop releases.

The central architectural change was not merely the addition of AI. It was the conversion of AI from an explanatory sidecar into a constrained participant in a deterministic debugging loop. The current design separates observation, diagnosis, proposal, authorization, mutation, and verification. A model may recommend a repair, but source mutation remains approval-gated and the result is rerun before being represented as verified. This paper reconstructs that evolution, identifies the design pressures behind each transition, records rejected or intentionally deferred ideas, and explains how security and human governance became first-class system properties.

Keywords: AI-assisted debugging, human-in-the-loop repair, sandboxed execution, LLM diagnostics, software verification, multi-language tooling, execution tracing, developer tools

1. Problem Origin

The project originated in a broader attempt to build an AI-driven creative production system. That work exposed a recurring systems problem: multiple specialized tools could generate useful outputs, yet failures across the pipeline remained fragmented across interfaces, runtimes, and external dependencies. The missing capability was not another generator. It was an orchestration layer able to observe failures, explain them, preserve context, and support repair.

AI Debugger Pro therefore emerged as infrastructure. The first implementation intentionally reduced the problem to a narrow deterministic loop. This was important: before an AI system can reason about repair, the surrounding software must establish what ran, what failed, and what evidence is trustworthy. The project's later phases repeatedly return to this principle.

2. Evolution at a Glance

Stage	Primary shift	System character
V0	Python parse + execute	Stateless deterministic checker
V1	LLM error interpretation	AI-assisted diagnosis
V2	Python/C/Java routing	Multi-language debugger
V3	Tkinter interaction	Desktop application
V4	History + diffs	Stateful debugging workspace
V5	Approval + verify + sandbox/context	Human-governed repair platform
V6	IDE polish + demos + releases	Recruiter/user-facing product prototype

3. V0 - Deterministic, Stateless Execution

The initial implementation used Python's AST parser for syntax validation and a subprocess-based execution path for valid source. The controller was linear: syntax check, execute if valid, capture stdout/stderr, return the result. A repeated CLI loop made the tool interactive, but runs were not persisted and no historical context crossed execution boundaries.

Design value. V0 established a deterministic evidence boundary. Syntax and runtime results came from language tooling rather than model inference.

Limitations. Python-only behavior, process-level rather than hardened isolation, no semantic explanation, no memory, and no repair lifecycle.

4. V1 - AI-Assisted Error Interpretation

V1 added LLM-based interpretation to the deterministic failure signal. Instead of replacing the executor, the AI layer consumed error evidence and generated explanations or suggested fixes. This was the project's first major conceptual split: the runtime establishes facts; the model interprets those facts.

This separation remains foundational in the current system. AI suggestions are optional, and the application retains deterministic offline guidance when an API key is absent. The architecture therefore avoids making basic execution dependent on model availability.

5. V2 - Multi-Language Execution

The next pressure was language heterogeneity. V2 introduced language-specific execution paths and central routing so that interpreted and compiled languages could share one debugging surface. The original paper records Python, C, and Java at this stage; the current implementation also supports C++. Compilation failures and runtime failures became distinct classes of evidence rather than being collapsed into a single generic error channel.

Architecturally, this moved the project away from a Python script toward a framework. Language selection became policy, while individual runners became replaceable implementation components.

6. V3 - Interactive Desktop Application

V3 introduced a Tkinter GUI, real-time interaction, and integrated AI feedback. The significance was larger than presentation. Event-driven interaction required the system to manage responsiveness, background work, and state transitions instead of assuming a blocking command-line exchange.

Later cleanup work explicitly strengthened responsive AI requests, helping prevent model calls from turning the interface into a frozen window. This was the beginning of a recurring engineering rule: intelligence is useful only if the surrounding application remains predictable.

7. V4 - Temporal State and Code Evolution

V4 added persistent execution history and unified diffs between recent code versions. This converted debugging from a sequence of isolated attempts into a temporal workflow. The user could now inspect not only the current failure, but how source and outcomes changed over time.

The original V0-V4 paper described this as a temporal memory layer. Importantly, it is not autonomous learning: stored history records executions and code evolution, but the system does not silently retrain itself or alter future behavior from accumulated sessions.

8. V5 - Human-Governed Repair and Engineering Hardening

The largest architectural jump occurred after the original V4 paper. The project stopped treating a suggested fix as the end of the debugging process and instead modeled repair as a controlled state machine: diagnose, propose, preview, approve or reject, apply, rerun, and record verification.

1. Observe Capture deterministic failure evidence	2. Diagnose Produce structured AI interpretation
3. Preview Render proposed source diff	4. Authorize Require explicit user approval
5. Apply Mutate source only after approval	6. Verify Rerun and record repair outcome

8.1 Approval as an Architectural Boundary

A repair proposal is not treated as permission to edit. The GUI exposes approve/reject controls, and the proposed patch is displayed as a unified diff before mutation. This makes user authority visible in both interface and control flow. The model can recommend; it cannot silently cross the authorization boundary.

8.2 Verification After Mutation

An approved repair is automatically rerun. The resulting state is recorded as verified or failed rather than assuming that syntactically plausible output is correct. Suggested regression tests extend the same principle: a repair claim should be tied to executable evidence.

8.3 Restricted Execution

Phase 3 engineering hardening made Docker the default backend and added defense-in-depth controls: no network access, resource limits, a read-only container root, dropped capabilities, and avoidance of inherited API secrets. Local execution remains possible only when deliberately selected. This does not create an absolute security boundary, but it materially improves the risk posture over the initial temporary-file subprocess model.

8.4 Exception-State and Project Context

The debugger now captures Python traceback frames with bounded local-variable representations and can discover a bounded set of project files for AI context. These changes address two different failure modes in AI debugging: too little context produces shallow diagnosis, while unbounded context creates privacy, cost, relevance, and prompt-size risks. Bounded collection is therefore both a quality mechanism and a safety mechanism.

8.5 Tooling Surface

The same core capabilities are exposed through CLI commands for execution, diagnosis, and project inspection, with starter VS Code extension commands providing an editor-integration path. This reduces coupling between the debugging engine and a single GUI.

9. V6 - Productization, Demonstrability, and Release Discipline

V6 focused on making the engineering legible to users and reviewers. The desktop interface was redesigned as a dark IDE-style workspace with a line-numbered editor, resizable history and result panels, dedicated output/diagnosis/diff/regression tabs, keyboard shortcuts, and explicit visual states for success, failure, warning, and loading.

A deterministic multi-demo recorder was then added. Rather than relying on hand-recorded idealized flows, the repository can regenerate demonstrations from the real desktop interface using offline data. Separate recordings show exception diagnosis, approval-gated repair, and regression verification. A GitHub Action can generate the same artifacts without an API key or Docker.

The final Phase 6 polish added recruiter-facing project positioning and tagged desktop release packaging for Windows, macOS, and Linux. These changes do not alter the core debugging algorithm, but they matter to reproducibility and evaluation: a technically strong system that cannot be installed, demonstrated, or inspected efficiently is harder to validate.

10. Architectural Principles That Emerged

Principle	Consequence
Deterministic evidence before probabilistic interpretation	Parsers, compilers, runtimes, tracebacks, and reruns establish facts; the model interprets them.
Human authority before mutation	A proposed patch remains a preview until the user explicitly approves it.
Verification after repair	Applied changes are rerun and recorded; plausibility is not treated as proof.
Bounded context	Project discovery and local-variable capture are constrained rather than indiscriminate.
Optional AI	Execution remains useful without a model key, preserving graceful degradation.
Interface independence	Core behavior is reachable through GUI and CLI, with an editor-integration path.
Security as layered risk reduction	Docker restrictions improve containment while documentation explicitly avoids claiming perfect isolation.

11. Rejected, Constrained, or Deferred Ideas

System evolution is also defined by what the project refuses to claim or automate. Several tempting directions were either rejected as unsafe assumptions or deferred until stronger evidence exists.

Idea	Disposition	Reason
Silent autonomous patching	Rejected in current design	Conflicts with explicit human authorization and makes model error directly mutative.
Model-only debugging	Rejected	Runtime and compiler evidence remain the factual substrate.
Unrestricted local execution by default	Rejected	Docker is the default; local execution requires deliberate opt-in.
Unlimited project/context ingestion	Constrained	Bounded discovery reduces irrelevant exposure and context inflation.
Full successful-statement tracing	Deferred	Current tracing prioritizes exception frames.
Arbitrary multi-service project execution	Deferred	Current prototype does not orchestrate arbitrary service graphs as one execution unit.
Cross-session self-learning	Deferred	History is persistence, not autonomous model adaptation.
Predictive / swarm debugging	Research direction	Requires a stronger evaluation and governance basis before product claims are justified.

12. Security Evolution

Security changed from an implementation caveat into an explicit architectural concern. The original executor relied on subprocess isolation and temporary files. The current default is restricted Docker execution with network removal, resource constraints, a read-only root, dropped capabilities, and secret-handling precautions. The README also states the correct boundary: no sandbox should be represented as absolute isolation, and the prototype should not be treated as a hosted untrusted-code execution service.

The repair workflow adds a second security dimension: integrity. Even a well-contained model can damage a codebase if its recommendations are applied automatically. Approval gating and post-change verification therefore protect the source tree, while sandboxing protects the execution environment. These are complementary controls, not substitutes.

13. Evaluation and Evidence

The original evaluation reported successful detection of syntax, runtime, and compilation failures, useful explanations for common errors, modular behavior across supported languages, responsive interaction, and temporal history/diff capabilities. The later repository strengthens the evidence surface with automated tests, Ruff linting, CI across Python 3.10 and 3.12, deterministic product demos, and a repair workflow whose result can be rerun.

However, this paper deliberately distinguishes demonstrated engineering from unmeasured research claims. The current repository supports a strong architectural case for human-governed AI repair; it does not yet establish comparative repair accuracy, statistically measured time-to-fix reduction, or superiority over existing IDE copilots. Those questions belong in a dedicated benchmark and evaluation paper.

14. Current Reference Architecture

The current system can be summarized as a sequence of evidence-preserving gates:

Editor / CLI	source + user intent
Syntax / language routing	deterministic validation and runner selection
Restricted execution	stdout, stderr, compile/runtime state, exception context
Structured diagnosis	AI interpretation + proposed repair
Diff preview	visible mutation candidate
Approval gate	human authorize / reject
Apply + rerun	deterministic post-change evidence
History / regression	persistent outcome + code evolution

15. Why the Evolution Matters

The most important result of V0-V6 is a change in the meaning of 'AI debugger.' At V1, AI meant better explanation. By V5, AI participates in a governed repair protocol. The system around the model now determines what evidence it receives, what output shape is acceptable, whether a proposed mutation is visible, who may authorize it, and how success is checked afterward.

That shift is strategically important for future multi-agent work. A swarm of agents without evidence boundaries, authorization rules, and verification loops merely scales uncertainty. AI Debugger Pro's current architecture supplies the control primitives that a future multi-agent debugging system would need: bounded context, structured roles, explicit state transitions, execution evidence, human override, and auditable outcomes.

16. Limitations

The current prototype remains intentionally bounded. Docker improves isolation but is not a perfect security boundary. Tracing emphasizes exception frames rather than every successful statement. Arbitrary multi-service applications are not executed as unified projects. AI diagnostic quality still depends on model capability and context quality. Suggested regression tests do not substitute for a mature project-specific test suite. Finally, the system has not yet been evaluated through a large controlled benchmark against expert developers or competing AI debugging products.

17. Conclusion

AI Debugger Pro evolved from a stateless Python execution helper into a multi-language, stateful, security-conscious debugging workspace with a human-governed repair loop. The decisive innovation is not autonomous code editing. It is controlled collaboration between deterministic software evidence, probabilistic AI reasoning, explicit human authorization, and executable verification.

This architecture creates a credible bridge toward more autonomous systems without pretending that autonomy itself is evidence of correctness. The next research step is therefore measurement: repair accuracy, regression prevention,

context-selection quality, security behavior, and human factors. Those evaluations can determine which parts of the present architecture deserve to become the foundation for adaptive and multi-agent debugging.

Appendix A - Repository Evidence Map

Evidence	Repository artifact / milestone
Original research paper	docs/paper.md and docs/AI Studio Debugger.pdf
Current architecture	README.md; core/analyzer.py; core/executor.py; core/sandbox.py; core/project.py; core/languages.py; core/ai_suggester.py; core/diagnostics.py; core/history.py; core/diff.py
Repair workflow	Phase 2 / PR #2: structured, approval-gated, verified repairs
Engineering platform	Phase 3 / PR #3: sandboxing, tracing, project context, CLI, and VS Code
Desktop redesign	Phase 4 / PR #4: polished dark IDE-style desktop interface
Deterministic demos	Phase 5 / PR #5 plus published generated demo assets
Release polish	Phase 6 / PR #7: recruiter-facing polish and desktop releases
Current CI	pytest + Ruff on Python 3.10 and 3.12 via GitHub Actions

Appendix B - Version Interpretation

The repository's recent engineering work is organized as numbered phases, while the original paper documents Versions 0-4. To preserve the historical narrative requested for this case study, this paper groups the post-V4 changes into V5 (repair/governance and engineering hardening) and V6 (interface, demonstrability, and release polish). This is a documentation convention for the evolution paper, not a claim that every grouped stage was published under a matching semantic version tag.

References

1. Bentley, T. R. *AI Debugger Pro: A Multi-Language Debugging Framework with LLM-Assisted Error Interpretation and Execution Tracing*. Project paper, 2026.
2. Tybent18/ai-debugger-pro, README and source tree, master branch, accessed September 2026.
3. AI Debugger Pro pull-request history: Phase 2 repair workflow; Phase 3 engineering platform; Phase 4 interface redesign; Phase 5 deterministic demo recorder; Phase 6 release polish.

Document role: system-evolution case study. A separate current-architecture paper should remain the authoritative description of the latest implementation; the original V0-V4 paper should be retained as a historical artifact rather than silently overwritten.